

Matrix Computations

Lecture 01: Logistics and Matrix Multiplications

Hongyuan Zha

School of Data Science
The Chinese University of Hong Kong, Shenzhen

SLAI

September 7, 2026



- 1 Introduction and Logistics
- 2 Matrix-vector and matrix-matrix multiplications
- 3 Vandermonde matrix and polynomial interpolation
- 4 Block elimination and Schur complement

Part 1. Introduction and Logistics

Lecture Information

Instructor:

- Hongyuan Zha, office hours: by appointment (send me a message via email or Wechat). zhahy@cuhk.edu.cn; hongyuanzha@slai.edu.cn

Office:

- CUHKSZ: Daoyuan Building 604
- SLAI: A620

Time:

- CUHKSZ: 01:30-02:50pm: TR
- SLAI: 9:30-12:15am, M

Location:

- CUHKSZ: Teaching C Building 308
- SLAI: B401

Teaching Assistant:

- CUHKSZ: Yiyang Lu; SLAI: Lixian Zhang

The goal of this course

- A comprehensive introduction to numerical linear algebra (NLA).
— NLA and matrix computations are used synonymously
- Present effective and state-of-the art NLA algorithms.
- Gain hands-on practice implementing NLA algorithms using Matlab/Python.
- Provide theoretical underpinnings of NLA algorithms through stability analysis, rounding error analysis and/or convergence analysis (for iterative methods)
- Learn how to effectively apply these techniques to new problems.

Prerequisites

- A basic linear algebra course (e.g., MAT2040, MAT2041, MAT2041A)
— at the level of Gil Strang's **Introduction to linear algebra and applications**
- A high-level programming language
- A certain level of **mathematical maturity**
— derivations and proofs
— this is also a course to enhance your "mathematical maturity"
— in many top universities, this is a required PhD course (at SLAI, you can use this course to satisfy your QE requirement)

Course Outline

Week	Content/ topic
	Linear algebra basics
1	Matrix-vector multiplication, orthogonality
2	Norms and SVD
	Linear least squares problems
3	Projections and QR factorization
4	Householder triangulation and linear least squares methods
	Conditioning and numerical stability
5	Condition numbers and floating-point arithmetics
6	Numerical stability, conditioning of least squares problems

Course Outline

Week	Content/ topic
	Linear systems
7	Gaussian elimination and pivoting
8	Cholesky factorization
	Eigenvalue problems
9	Hessenberg reduction and QR algorithms
10	QR algorithms with shifts, and SVD computation
	Iterative methods
11	Arnoldi iteration and GMRES
12	Lanczos methods and eigenvalue problems
13	Preconditioning
14	Random linear least squares problems

Textbook:

- “Numerical Linear Algebra” (1997) by L. N. Trefethen, D. Bau III

Other readings:

- “Applied Numerical Linear Algebra” (2017) by J. Demmel
- “Matrix Computations” (4th edition, 2013) by G. Golub, C. Van Loan
- “Matrix Algorithms” (two volumes, 1998) by G.W. Stewart
- “Iterative Methods for Sparse Linear Systems” (2nd edition, 2003) by Y. Saad

Assessment Scheme

- Assignments 20% (4 homework assignments)
- Mid-term 35%
- Final exam 35%
- Attendance 10%

Why is NLA useful?

CS algorithm courses deal with discrete objects (lists, trees, graphs, ...). For continuous computations, matrix computations serve as the starting point. NLA provides **foundations** for

- Numerical solution of differential equations or integral equations: sparse or structured large-scale linear systems
- Numerical optimization: second order methods, or preconditioners for first-order methods
- Statistics and machine learning: regression problems, LoRA/SFT

Types of NLA methods

- **Direct methods**: based on matrix factorizations such as QR factorization, LU factorization
 - dense matrices
 - sparse matrices (matrices with many zero entries): preserving sparsity by reducing fill-ins in the intermediate steps
- **Iterative methods**: the matrix A is accessed via matrix-vector multiplication, rather than directly operating on A
 - sparse matrices
 - structured matriceswhere matrix-vector multiplication can be computed cheaply
- **Randomized** numerical linear algebra (RandNLA): use of randomization for NLA, close connection with TCS and ML

Part 2. Matrix-vector and matrix-matrix multiplications

Some notations

- For an $m \times n$ matrix A with **complex** entries, we write $A \in C^{m \times n}$. Often we use $A \in R^{m \times n}$ for matrices with **real** entries. Vectors are special cases of matrices
- If $b = Ax$ (the result of a matrix-vector multiplication), and $A = [a_{ij}]$, $b = [b_i]$, $x = [x_i]$ (written in component-wise fashion), we write

$$b_i = \sum_{j=1}^n a_{ij}x_j, \quad i = 1, \dots, m$$

- If we write A column-wise: $A = [A_1, A_2, \dots, A_n]$, we can also write

$$b = \sum_{j=1}^n A_j x_j, \quad a_i \in C^m, x_i \in C$$

- a **linear combination** of the columns of A

Column space of a matrix

- $\mathcal{C}(A)$, the column space of A , is the subspace spanned by the columns of A , also denoted by $\text{span}(A)$
- Given A and b , the linear system $Ax = b$ has a solution iff $b \in \mathcal{C}(A)$
- If $Ax = b$ has a solution, then the solution is unique iff A has full column-rank, or the columns of A is **linearly independent**. Or equivalently, the null space of A , $\mathcal{N}(A) = \{0\}$, the trivial subspace
- When $b \notin \mathcal{C}(A)$, $Ax = b$ has no solution. We may seek an approximate solution by solving (here $\|\cdot\|_2$ denotes vector 2-norm)

$$\min_{x \in \mathbb{C}^n} \|b - Ax\|_2,$$

leading to *linear least squares* problems, or linear regression problems, or linear supervised learning problems (connection to statistics and ML)

Part 3. Vandermonde matrix and polynomial interpolation

An Example: Polynomials and Vandermonde matrix

- Alexandre-Théophile Vandermonde (1735 – 1796)
- Matrix-vector multiplication is related to polynomial interpolation
- Given a polynomial of degree $< n$ in unknown $x \in C$,

$$p(x) = c_0 + c_1x + \cdots + c_{n-1}x^{n-1}$$

c_i are the coefficients of p , and if $c_{n-1} \neq 0$, the degree of p is $n - 1$

- Also given are m real numbers $x_1, \dots, x_m$. Consider evaluating the polynomial p at those m points

$$p(x_1), \dots, p(x_m)$$

i.e., what are the values of the polynomial p at those points

Vandermonde matrix and interpolation

- This task of evaluation is equivalent to a matrix-vector multiplication

$$\begin{bmatrix} p(x_1) \\ p(x_2) \\ \vdots \\ p(x_m) \end{bmatrix} = \begin{bmatrix} 1 & x_1 & x_1^2 & \cdots & x_1^{n-1} \\ 1 & x_2 & x_2^2 & \cdots & x_2^{n-1} \\ \vdots & \vdots & \vdots & & \vdots \\ 1 & x_m & x_m^2 & \cdots & x_m^{n-1} \end{bmatrix} \begin{bmatrix} c_0 \\ c_1 \\ \vdots \\ c_{n-1} \end{bmatrix}$$

or we write $p = Ac$

- *Interpolation.* Given m real values $y_1, \dots, y_m$, find a polynomial p such that $p(x_i) = y_i, i = 1, \dots, m$
- The above is equivalent to $y = Ac$, where A is the so-called Vandermonde matrix. We can also consider the related linear least squares problem

$$\min_c \|y - Ac\|_2$$

General basis functions

- Polynomials are linear combinations of the monomials

$$1, x, x^2, \dots, x^{n-1}$$

consider as a set of basis functions of the set of polynomials of degree $< n$. (A polynomial is a linear combination of the monomials)

- For $x \in R^d$, we can consider more general basis functions

$$B_1(x), B_2(x), \dots, B_n(x)$$

and their linear combinations

$$f(x) = c_1 B_1(x) + c_2 B_2(x) + \dots + c_n B_n(x)$$

General basis functions

- Given $y_i, i = 1, \dots, m$, we can again similarly consider $f(x_i) = y_i$, or the least squares version of the problem

$$\begin{bmatrix} f(x_1) \\ f(x_2) \\ \vdots \\ f(x_m) \end{bmatrix} = \begin{bmatrix} B_1(x_1) & B_2(x_1) & \cdots & B_n(x_1) \\ B_1(x_2) & B_2(x_2) & \cdots & B_n(x_2) \\ \vdots & \vdots & \vdots & \vdots \\ B_1(x_m) & B_2(x_m) & \cdots & B_n(x_m) \end{bmatrix} \begin{bmatrix} c_1 \\ c_2 \\ \vdots \\ c_n \end{bmatrix}$$

- One key question in statistics and ML is where the basis functions come from
 - one answer is that they are based on deep neural networks learned from data (rather than hand-crafted based on prior experiences)

Matrix multiplications and block matrices

- Given two matrices $A \in R^{m \times n}$ and $B \in R^{n \times p}$, their product is defined (in terms of matrix-vector multiplications) as

$$C = AB = [Ab_1, \dots, Ab_p] \in R^{m \times p}, \quad B = [b_1, \dots, b_p]$$

- Block matrices**, for example, A partitioned as 2×2 blocks

$$\begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix}$$

- Multiplication of two block matrices

$$\begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix} = \begin{bmatrix} A_{11}B_{11} + A_{12}B_{21} & A_{11}B_{12} + A_{12}B_{22} \\ A_{21}B_{11} + A_{22}B_{21} & A_{21}B_{12} + A_{22}B_{22} \end{bmatrix}$$

The dimensions of the submatrices need to be compatible!

Matrix multiplication and block matrices

- Given two matrices $A \in R^{m \times n}$ and $B \in R^{n \times p}$, their product

$$C = AB = [Ab_1, \dots, Ab_p] \in R^{m \times p}, \quad B = [b_1, \dots, b_p]$$

- Let's consider several other ways of expressing AB depending on how A and B are partitioned
- Partition A row-wise

$$A = \begin{bmatrix} A_1^T \\ \vdots \\ A_m^T \end{bmatrix}, \quad AB = \begin{bmatrix} A_1^T B \\ \vdots \\ A_m^T B \end{bmatrix}$$

Matrix multiplication and block matrices

- Partition A row-wise and B column-wise

$$AB = \begin{bmatrix} A_1^T \\ \vdots \\ A_m^T \end{bmatrix} [b_1, \dots, b_p] = \begin{bmatrix} A_1^T b_1 & \cdots & A_1^T b_p \\ \vdots & & \vdots \\ A_m^T b_1 & \cdots & A_m^T b_p \end{bmatrix}$$

a set of inner-products $A_i^T b_j$, the usual way AB is computed by hand

- Partition A column-wise and B row-wise

$$AB = [a_1, \dots, a_n] \begin{bmatrix} B_1^T \\ \vdots \\ B_n^T \end{bmatrix} = \sum_{i=1}^n a_i B_i^T$$

a sum of rank-one matrices, where $a_i B_i^T$ is a rank-one matrix

Part 4. Block elimination and Schur complement

Punching a *hole* in a block matrix via matrix multiplication

- Matrix-matrix multiplication is fundamental in matrix factorizations
- Given a 2×2 block matrix A

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix}$$

assume A_{11} is nonsingular. We want to *eliminate* A_{21} , i.e., set it to a zero block by pre-multiplying with a special nonsingular matrix

- It turns out a block lower triangular matrix will do

$$\begin{bmatrix} I & 0 \\ -L_{21} & I \end{bmatrix} \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} = \begin{bmatrix} A_{11} & A_{12} \\ -L_{21}A_{11} + A_{21} & -L_{21}A_{12} + A_{22} \end{bmatrix}$$

- Set $-L_{21}A_{11} + A_{21} = 0$, we obtain $L_{21} = A_{21}A_{11}^{-1}$

Schur complement

- Issai Schur (1875 – 1941)
- It turns out

$$\begin{bmatrix} I & 0 \\ -L_{21} & I \end{bmatrix}^{-1} = \begin{bmatrix} I & 0 \\ L_{21} & I \end{bmatrix}$$

- Therefore,

$$\begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} = \begin{bmatrix} I & 0 \\ L_{21} & I \end{bmatrix} \begin{bmatrix} A_{11} & A_{12} \\ 0 & -A_{21}A_{11}^{-1}A_{12} + A_{22} \end{bmatrix}$$

A block *LU factorization* of A

- The trailing $(2, 2)$ block $-A_{21}A_{11}^{-1}A_{12} + A_{22}$ is called a *Schur complement* (of A_{11} with respect to A)